

# Build Plan

# Final Build Plan — Local AI Coding Assistant

## 1. Local AI Model Setup (The Brain)

We will use an open-source coding-focused LLM that runs locally instead of using external APIs.

The exact model will be decided later (for example Qwen Coder, DeepSeek Coder, or another suitable model).

Our system will:

- load the model locally
- send tasks and information to it
- receive reasoning, decisions, and responses from it

We are **not building the AI model itself**. We are building the system around it that allows the model to work like a developer.

---

## 2. Local Inference Layer (Model Runner)

We will use/build the layer that allows our Python application to communicate with the local LLM.

Its purpose:

- load the model into memory
- handle requests from our application
- manage generation of responses
- optimize model execution on available hardware

Initially this can use existing local inference engines (such as Ollama/llama.cpp/vLLM), while our main focus remains the agent system.

---

### 3. Agent Controller (Main Python Application)

This will be the core of our project.

It will manage the entire AI workflow.

The agent controller will:

- receive developer requests
- decide what needs to be done
- communicate with the LLM
- manage tools
- maintain context
- track progress
- verify results

This is where the intelligence of the application comes together.

---

### 4. Planning + ReAct System

We will build an agent workflow where the AI does not immediately generate an answer.

Instead:

- understand the goal
- create a plan
- decide the next action
- use available tools
- observe the result
- decide the next step

This allows the AI to behave more like a software developer working through a problem.

---

### 5. Repository Understanding System

We will build a system that allows the AI to understand existing projects.

It will include:

## **Repository Scanner**

Analyzes:

- project structure
  - files
  - dependencies
  - configuration
  - technologies used
- 

## **AST-Based Code Indexer**

Instead of splitting code randomly, we will analyze code structure.

Using AST parsing, the system will understand:

- functions
- classes
- components
- modules
- relationships between code sections

This allows better retrieval of relevant code.

---

## **Search and Retrieval System**

When the AI needs information, the system will find the most relevant parts of the project and provide them to the model.

This allows the AI to work with large projects without loading everything.

---

## **6. Tool System (AI Capabilities)**

We will build tools that the AI can use.

Examples:

## **File Tools**

Allow the AI to:

- read files
  - create files
  - edit files
  - apply changes
- 

## **Code Search Tools**

Allow the AI to:

- search the project
  - find functions/classes
  - locate related code
- 

## **Execution Tools**

Allow the AI to:

- run commands
  - run tests
  - inspect errors
  - verify changes
- 

## **Git Tools**

Allow the AI to:

- view modifications
- track changes

- prepare developer reviews
- 

## **7. Safe Code Modification System**

The AI will not rewrite entire files unnecessarily.

We will build a controlled editing system where the AI produces:

- search/replace changes
- patches
- structured modifications

The Python application will validate and apply these changes.

This makes editing faster and safer.

---

## **8. State Management System**

Before the AI changes anything, the system will store the current state.

Purpose:

- prevent accidental damage
- allow rollback
- recover from failed attempts

The system will track:

- files changed
  - actions performed
  - test results
  - agent progress
- 

## **9. Testing and Verification Loop**

After making changes, the agent will verify its work.

The workflow:

- modify code
- run tests/build
- analyze errors
- fix problems
- retry

The goal is for the AI to improve its own output.

---

## 10. Context Management System

Because local models have limited memory, we need a context manager.

It will control:

- how much code is sent to the model
- conversation history
- tool outputs
- previous actions

It will summarize unnecessary information and keep only what is important.

---

## 11. Team and Project Memory

For team usage, we will add memory about:

- project architecture
- development practices
- preferred technologies
- previous decisions

This allows the AI to become familiar with the project over time.

---

# Final System Overview

The complete application will work like this:

Developer gives a task

- Agent Controller understands the request
  - Planner/ReAct decides what to do
  - Repository System provides relevant code knowledge
  - Tools allow the AI to interact with the project
  - LLM provides reasoning and decisions
  - Changes are safely applied
  - Tests verify the result
  - Developer reviews the outcome
-



# Execution Plan

# Execution Plan — Building the Local AI Coding Assistant

## Phase 0 — Local LLM Setup (Before Phase 1)

**Goal:** Get a local model running and make sure our Python application can communicate with it.

We build:

- local inference setup
- model loading
- basic prompt/response communication
- model configuration

Example:

Python application sends:

"Explain this code file"

Local LLM responds.

**Testing:**

- Can the model run locally?
- Is response quality acceptable?
- Is inference speed usable?
- Does hardware handle the model?

**Milestone:** Python application can talk to a local coding model.

---

## Phase 1 — Build the Repository Understanding System

**Goal:** Make the assistant capable of understanding and explaining existing projects.

**Build:**

- Repository scanner
- File and dependency analyzer
- AST-based code indexing system
- Code search/retrieval system
- Basic project memory

**Expected capability:**

The developer can give the assistant a project and ask:

- What is this application?
- How is it structured?
- Where is a specific feature implemented?
- How do different parts connect?

**Testing:**

Use different sample projects:

- simple Python project
- React + backend project
- HTML/CSS website
- larger open-source repository

Check:

- Does it identify technologies correctly?
- Does it find relevant files?
- Are explanations accurate?
- Does it understand relationships between files?

**Milestone:** Assistant can analyze a repository and explain it like a developer who studied the codebase.

---

## Phase 2 — Build the Tool System

**Goal:** Give the assistant the ability to interact with the project.

**Build:**

- File reading tools
- Code search tools
- File creation/editing tools
- Command execution tools
- Git integration

**Method:**

- The AI should not directly modify anything.
- The Python agent controls all actions through tools.

**Testing:**

Give tasks like:

- Find a configuration file
- Locate a function
- Explain a bug location
- Create a new file

Check:

- Does the assistant choose the correct tools?
- Does it access the correct files?
- Does it avoid unnecessary actions?

**Milestone:** Assistant can safely inspect and interact with a project.

---

## **Phase 3 — Build the Coding Agent (Planner + ReAct)**

**Goal:** Make the assistant capable of completing development tasks.

**Build:**

- Task planner
- ReAct reasoning loop
- Tool decision system

- Task tracking

The workflow becomes:

Understand → Plan → Act → Observe → Continue

### **Testing:**

Give controlled tasks:

- Add a small feature
- Modify an existing function
- Update a component
- Fix a simple bug

Check:

- Did it create a reasonable plan?
- Did it choose the correct actions?
- Did it complete the task?

**Milestone:** Assistant can complete small coding tasks independently.

---

## **Phase 4 — Build Safe Code Modification and Recovery**

**Goal:** Make the assistant reliable when changing real projects.

### **Build:**

- Search/replace editing system
- Patch-based modifications
- File snapshots
- Rollback system

The assistant should never blindly rewrite large files.

### **Testing:**

Create failure scenarios:

- incorrect code change
- failed build
- broken test
- interrupted execution

Check:

- Can changes be reverted?
- Can the agent recover?
- Are developer changes protected?

**Milestone:** Assistant can modify code safely.

---

## Phase 5 — Build Testing and Debugging Ability

**Goal:** Allow the assistant to validate and improve its own work.

**Build:**

- Test execution tools
- Error analysis
- Debugging loop
- Build verification

Workflow:

Change → Test → Analyze → Fix

**Testing:**

Give tasks where:

- first attempt fails
- tests reveal problems
- debugging is required

Check:

- Can it understand errors?
- Can it fix mistakes?

- Does it stop when stuck?

**Milestone:** Assistant can implement changes and verify them.

---

## Phase 6 — Improve Context and Memory Management

**Goal:** Make the assistant work on larger projects.

**Build:**

- Context budget manager
- Conversation compression
- Improved project memory
- Better retrieval ranking

**Testing:**

**Use:**

- large repositories
- long conversations
- complex tasks

**Check:**

- Does it retain important information?
- Does it avoid sending unnecessary code?
- Does performance remain stable?

**Milestone:** Assistant can handle realistic development projects.

---

## Phase 7 — Team Features and Developer Experience

**Goal:** Make it usable by a development team.

**Build:**

- Shared project memory
- User preferences
- Approval workflows
- Change summaries
- Web interface

### Testing:

Simulate team workflows:

- multiple developers
- shared repositories
- different tasks

Check:

- Is collaboration smooth?
- Are changes understandable?
- Is the assistant predictable?

**Milestone:** Assistant becomes a practical team development tool.

---

So the actual architecture timeline is:

Phase 0:

Local LLM running

↓

Phase 1:

LLM understands repositories

↓

Phase 2:

LLM uses tools

↓

Phase 3:

LLM performs coding tasks





Phase 4+:

Make LLM reliable, safe, and team-ready

# Implementation Choices

# LLM Options Summary for Local AI Coding Assistant

## Option 1 — Qwen Coder Series (Recommended Starting Point): Qwen3-Coder / newer Qwen Coder models

**Best fit for our project.**

Why:

- designed for coding tasks
- strong programming language support
- better suited for agent workflows
- good at multi-step coding tasks
- works well with tool-based systems

Good for:

- repository understanding
- code changes
- debugging
- coding assistant workflows

---

## Hardware requirements (approximate)

### Small models (around 7B)

Good for:

- development/testing
- personal laptops

Requirements:

- 16GB RAM minimum
- GPU optional

- better with 8GB+ VRAM

Expected performance:

- usable
  - faster
  - weaker reasoning
- 

### **Medium models (around 14B) - Recommended balance.**

Requirements:

- 32GB RAM recommended
- GPU with around 12–16GB VRAM preferred
- can run quantized locally

Expected performance:

- strong coding ability
  - better planning
  - suitable for real developer use
- 

### **Larger models (30B+)**

Requirements:

- powerful workstation
- 24GB+ VRAM GPU
- or server setup

Expected performance:

- closer to premium coding assistants
  - slower and more expensive
- 

## **Option 2 — Qwen3.5 / Qwen3.6 general models**

Interesting because newer Qwen generations appear to move toward more general agent capability rather than having completely separate "coder-only" behavior.

Potential advantage:

A coding assistant does more than write code:

- reads documentation
- reasons about architecture
- understands screenshots/UI
- handles natural language requests

A broader model may perform better there.

---

## **Option 3 — DeepSeek Coder Series**

Strong alternative.

Strengths:

- excellent code generation
- good reasoning
- strong debugging ability

Good for:

- complex programming tasks
- backend development
- algorithmic problems

Hardware requirements are similar:

- 7B class → normal developer machine
  - 14B class → workstation
  - larger versions → server hardware
- 

## **Recommended Development Approach**

Do not lock the system to one model.

Build the application so the model can be swapped:

Python Agent



Model Interface



Qwen / DeepSeek / Other Local Model

This allows us to improve the model later without rewriting the application.

---

## Suggested Starting Choice

For the first serious prototype:

### Model:

- Qwen Coder medium-size model (around 14B)

### Runtime:

- Ollama initially

### Hardware target:

- Developer workstation
- 32GB RAM recommended
- GPU preferred

Later:

- move to larger models
  - use vLLM
  - deploy on a team GPU server
-

## Simple Decision Table

| Use Case                 | Recommended Model Size | Hardware                     |
|--------------------------|------------------------|------------------------------|
| Testing/prototype        | 7B                     | 16GB RAM laptop              |
| Individual developer use | 14B                    | 32GB RAM + GPU preferred     |
| Professional team usage  | 30B+                   | GPU workstation/server       |
| Maximum capability       | 70B+                   | Dedicated GPU infrastructure |

---

**Overall recommendation:** Start with a Qwen Coder model around the 14B range because it gives the best balance between local practicality and the reasoning ability required for a coding agent. The architecture should remain model-independent so we can upgrade later.

---

## Quantization Summary

### What quantization does (simple explanation)

A model stores its knowledge using billions of numerical values called weights.

A normal model might store them using:

- FP32 (32-bit numbers)
- FP16 (16-bit numbers)

Quantization compresses these values into smaller formats:

- INT8
- INT4
- etc.

So instead of:

"Store this model with maximum precision"

we say:

"Store a smaller approximation that is close enough."

The result:

- smaller model size
  - lower RAM/VRAM usage
  - faster loading
  - easier local deployment
- 

## Why it matters for our project

Without quantization:

Example:

Qwen Coder 14B

might require:

- large RAM
- significant GPU memory

With quantization:

The same model can potentially run on:

- a developer workstation
- smaller GPU
- even CPU-only setups (with slower performance)

This directly helps our goal:

A local coding assistant that teams can run themselves.

---



## Does quantization reduce performance?

Yes, but the amount depends on the level.

### **FP16 (full quality)**

Best quality.

Downside:

- highest hardware requirements
- 

### **INT8**

Usually very small quality impact.

Good balance.

---

### **INT4**

Big reduction in size.

Usually:

- still very usable
- small reasoning loss
- occasional coding mistakes increase

For a coding agent, this is where we need to be careful.

---

## Example trade-off

Imagine:

Qwen Coder 14B

Full precision:

- excellent reasoning
- needs stronger hardware

Quantized:

- slightly less capable
- runs locally much easier

For a coding assistant, a quantized 14B model may still outperform a smaller unquantized model.

Example:

Better:

**14B INT4**

vs

**7B FP16**

because the larger model retains more reasoning ability.

---

## Where quantization fits in our architecture

It should be part of the LLM layer.

Our design becomes:

Developer



Agent Controller



LLM Interface



Local Inference Engine



## Quantized Coding Model

The agent itself does not need to know whether the model is:

- FP16
- INT8
- INT4

It just communicates with it.

---

## Recommended approach for our project

### Development phase

Start with:

- Qwen Coder / DeepSeek Coder
- quantized version

Probably:

- 4-bit quantization

Reason:

We want more developers to be able to run it.

---

### Benchmark phase

Compare:

- FP16 model
- INT8 model
- INT4 model

Test:

- repository explanation
- code changes
- debugging
- tool usage

Choose based on actual performance.

---

## One important consideration

For coding agents, model quality is not only the model.

Our architecture already improves performance through:

- AST-based retrieval
- better context management
- controlled editing
- testing loops

This means quantization impact can be reduced because we are not asking the model to solve everything from memory.

We are giving it:

- correct files
  - structured context
  - tools
  - feedback
- 

## Recommendation

Include quantization as a core project consideration.

The practical setup would likely be:

**Prototype:**

- Qwen Coder 7B/14B
- 4-bit quantized

- Ollama/llama.cpp

**More powerful deployment:**

- Qwen Coder 14B/32B
- 8-bit or FP16
- GPU server